ЛАБОРАТОРНАЯ РАБОТА №2

«ИССЛЕДОВАНИЕ МЕХАНИЗМА НАСЛЕДОВАНИЯ»

1. Цель работы

Исследование основных средств создания базового и производного классов. Исследование особенностей вызова методов производного класса при простом наследовании.

2. Постановка задачи и вариант задания

Описать иерархию классов, состоящую из трёх уровней (рис. 2.1). Каждый уровень реализовать как один C++ класс, используя средства языка для организации иерархии. Первый уровень – базовый класс, минимум с одним информационным полем. Второй и третий уровень – производные классы, минимум с двумя информационными полями. Для каждого класса необходимо определить по два конструктора (с параметрами и без), а также методы ввода и вывода данных и метод, переопределяемый в производных классах (с одинаковым названием и разной реализацией).



Рисунок 2.1 – Вид иерархии

В основной программе необходимо создать 2 объекта производного класса третьего уровня (вызов конструктора по умолчанию и вызов конструктора с параметрами), осуществить обращение к доступным элементам иерархии (обязательно осуществить вызов переопределенного метода и вызов этого же

метода у классов первого и второго уровней). Результаты работы вывести на экран и внести в отчет

Вариант 4: Класс первого уровня – Устройство; второго – Часы; третьего – Песочные.

ХОД РАБОТЫ

3. Листинг кода на языке C++

```
#include <iostream>
#include <iomanip>
#include <string>

using namespace std;

class device {
    protected:
        string name;

    public:
        device() : name("Устройство") {
            cout << "  device(): конструктор по умолчанию\n";
        }
        device(string n) : name(n) {
            cout  <<  "    device(\""  <<  n  <<  "\"):  конструктор  с
параметром\n";
        }
        ~device() {
            cout << "  ~device(): " << name << "\n";
        }

        void show() {
            cout << "  [device] Название: " << name << "\n";
        }
        string getName() { return name; }
};

class clock : public device {
    protected:
        int hours;

    public:
        clock() : device("Часы"), hours(0) {
            cout << "  clock(): конструктор по умолчанию\n";
        }
        clock(const string& n, int h) : device(n), hours(h) {
            cout << "  clock(\"" << n << "\", " << h << "): конструктор
с параметром\n";
        }
        ~clock() {
            cout << "  ~clock(): " << name << "\n";
        }
        void show() {
            device::show();
            cout << "  [clock]  Показание: " << hours << " ч.\n";
        }
        int getHours() { return hours; }
```

```
};

class sandClock : public clock {
    private:
        double sandVolume;

    public:
        sandClock() : clock("Песочные часы", 0), sandVolume(0.0) {
            cout << "  sandClock(): конструктор по умолчанию\n";
        }
        sandClock(const string& n, int h, double v) : clock(n, h),
sandVolume(v) {
            cout << "  sandClock(\"" << n << "\", " << h << ", " << v
                 << "): конструктор с параметром\n";
        }
        ~sandClock() {
            cout << "  ~sandClock(): " << name << "\n";
        }
        void show() {
            clock::show();
            cout << "  [sandClock] Объём песка: " << sandVolume << "
г.\n";
        }
        double getSandVolume() const { return sandVolume; }
};

int main() {
    cout << "=== 1. Создание объекта obj1 (конструктор по умолчанию)
===\n";
    sandClock obj1;

    cout << "\n=== 2. Создание объекта obj2 (конструктор с параметрами)
===\n";
    sandClock obj2("Песочные часы на 5 минут", 0, 250.0);

    cout << "\n=== 3. Вызов переопределённого метода obj1.show()
===\n";
    obj1.show();

    cout << "\n=== 4. Вызов переопределённого метода obj2.show()
===\n";
    obj2.show();

    cout << "\n=== 5. Явный вызов метода УРОВНЯ 2: obj2.clock::show()
===\n";
    obj2.clock::show();

    cout << "\n=== 6. Явный вызов метода УРОВНЯ 1: obj2.device::show()
===\n";
    obj2.device::show();

    cout << "\n=== 7. Полиморфный вызов через указатель на device*
===\n";
```

```
    device* p = &obj2;
    p->show();

    cout << "\n=== 8. Доступ к protected/private через геттеры ===\n";
    cout << "  Имя:   " << obj2.getName()       << "\n";
    cout << "  Часы:  " << obj2.getHours()      << "\n";
    cout << "  Песок: " << obj2.getSandVolume() << " г\n";

    cout << "\n=== Завершение программы ===\n";
    return 0;
}
```

4. Тестирование и отладка программы

На рисунке 2.2 отображены результаты работы кода согласно всем условиям по заданию.

```
=== 1. Создание объекта obj1 (конструктор по умолчанию) ===
  device("Песочные часы"): конструктор с параметром
  clock("Песочные часы", 0): конструктор с параметром
  sandClock(): конструктор по умолчанию

=== 2. Создание объекта obj2 (конструктор с параметрами) ===
  device("Песочные часы на 5 минут"): конструктор с параметром
  clock("Песочные часы на 5 минут", 0): конструктор с параметром
  sandClock("Песочные часы на 5 минут", 0, 250): конструктор с параметром

=== 3. Вызов переопределённого метода obj1.show() ===
  [device] Название: Песочные часы
  [clock]  Показание: 0 ч.
  [sandClock] Объём песка: 0 г.

=== 4. Вызов переопределённого метода obj2.show() ===
  [device] Название: Песочные часы на 5 минут
  [clock]  Показание: 0 ч.
  [sandClock] Объём песка: 250 г.

=== 5. Явный вызов метода УРОВНЯ 2: obj2.clock::show() ===
  [device] Название: Песочные часы на 5 минут
  [clock]  Показание: 0 ч.

=== 6. Явный вызов метода УРОВНЯ 1: obj2.device::show() ===
  [device] Название: Песочные часы на 5 минут

=== 7. Полиморфный вызов через указатель на device* ===
  [device] Название: Песочные часы на 5 минут

=== 8. Доступ к protected/private через геттеры ===
  Имя:   Песочные часы на 5 минут
  Часы:  0
  Песок: 250 г

=== Завершение программы ===
  ~sandClock(): Песочные часы на 5 минут
  ~clock(): Песочные часы на 5 минут
  ~device(): Песочные часы на 5 минут
  ~sandClock(): Песочные часы
  ~clock(): Песочные часы
  ~device(): Песочные часы
```

Рисунок 2.2 – Результат работы программы на языке C++

ВЫВОД

В ходе выполнения лабораторной работы были изучены основы средств создания базового и производного класса, а также исследованы особенности механизма простого наследования, а в частности особенности вызова методов производного класса при простом наследовании.